

Nature of Development Testing

The following might seem a bit obvious (to such experienced devs) but worth stating anyway...

- When you test, you execute a program
 - You “exercise” it using synthetic data
 - Checking the results for errors or anomalies
 - Aim of testing is to identify presence of defects
 - A successful test is one that finds a defect !
- “Absence of evidence is not evidence of absence”

Example

Consider a function that,
if given a day, month and year,
would tell you what day of the week it was
Mon, Tues, Weds, Thurs, Fri, Sat, Sun

(Zeller's congruence)

How do we know if it is **completely** correct ?

Correctness & Completeness

- We could run program with a few arbitrary dates
- But that probably won't reveal many errors
- It isn't methodical or systematic enough
- How to ensure complete & comprehensive test ?
- What we need is a proper, organised strategy...

Coverage versus Practicality

- Functions have finite number of input parameters
- Complete black-box testing would try every single possible combination of such input values
- This obviously isn't always practical...
- For the previous “day of the week” function, 100 years would require around 37k combinations !
- We need to identify a **sample set** of test cases
- To ensure full coverage, but without having to exhaustively try all combinations

Equivalence Partitioning

A technique to help systematic selection of test cases

An equivalence partition is:

“A cluster of input values for which a program should behave in the same way”

An example partition might be set of all +ve numbers (up to max allowable in the programming language)

The number 54 is **likely** to have a **similar** effect to 55 !

Selecting Test Data

For each partition, select upper & lower boundary
(Boundary values can be overlooked by coders)

Also choose a data value from middle of partition
Should be representative of main body of values

In this way, we end up testing each partition, rather than every single possible value or combination

Example Equivalence Partitions

Imagine we had a function to convert a grade (as a percentage) into a degree classification
(1st, 2.1, 2.2, 3rd, Fail)

The equivalence partitions might be as follows:

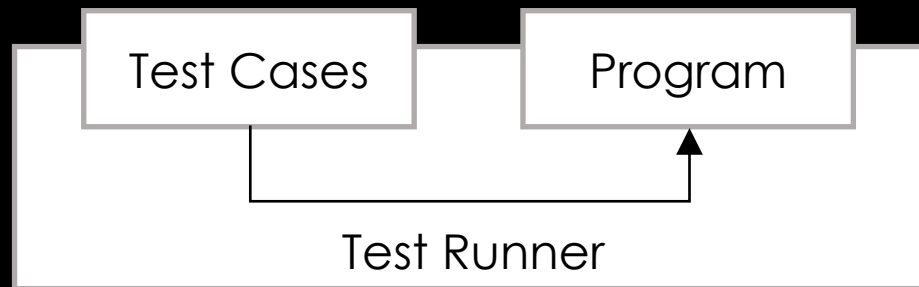
...	-1	0	1	39	40	49	50	59	60	69	70	100	101	...
-----	----	---	---	----	----	----	----	----	----	----	----	-----	-----	-----

The test case values might then be:

-10, -1, 0, 1, 26, 39, 40, 43, 49, 50, 56, 59, 60, 63, 69, 70, 80, 100, 101

JUnit

- JUnit is a simple tool to aid unit testing
- Provides a framework which developers use to write and run their own test cases:



- When using JUnit, you write test cases in Java !
- So we can write loops to test many values

Assertions

- JUnit uses “Assertions” to test the code
- Allow us to state what *should* be the case
- If assertions do not hold, JUnit’s logging mechanisms reports failures
- Various types of assertion are available
- The most commonly used are:
 - assertEquals(expected, actual)
 - assertTrue(condition)
 - assertFalse(condition)
 - assertThat (value, matchingFunction)

Examples

Given three integer variables:

```
int x = 1;    int y = 10;    int z = 100;
```

What are the outcomes of:

PASS assertEquals(1, x);

FAIL assertEquals(x, z);

PASS assertTrue(Math.sqrt(y) > x);

PASS assertFalse(x > y);

FAIL assertFalse(z != y);

JUnit integration with Spring

Pom dependency (comes as default from initializr):
spring-boot-starter-test

Add annotations to java files in /src/test/java:

@SpringBootTest.....for the whole class

@Test.....for each test method

“import static” each assertion method you want to use:

import static org.junit.jupiter.api.Assertions.assertEquals;

import static org.junit.jupiter.api.Assertions.assertTrue;

To run the tests ? Just build the project - as it should be !!!

Code Coverage

- A measure of % of a system “exercised” by tests
- The aim is to maximise the level of coverage
- If big chunks of the code are missed by testing...
...we haven’t tested it properly !
- The **higher** the coverage, the **more** confident we are that the system operate as expected

JaCoCo (Java Code Coverage)

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxt
 Palindrome		21%		17%	3	
Total	30 of 38	21%	5 of 6	17%	3	

```
public class Palindrome {  
    public boolean isPalindrome(String inputString) {  
        if (inputString.length() == 0) {  
            return true;  
        } else {  
            char firstChar = inputString.charAt(0);  
            char lastChar = inputString.charAt(inputString.length() - 1);  
            String mid = inputString.substring(1, inputString.length() - 1);  
            return (firstChar == lastChar) && isPalindrome(mid);  
        }  
    }  
}
```

Not all branches have been exercised

The else part wasn't run !